Fundamentos Analíticos y Datos

Módulo Avanzado: Pandas y Manejo Tabular

1. Estructuras Centrales de Pandas

PREGUNTA **¿Cuáles son las bases estructurales de Pandas y cómo difieren de los tipos nativos?**

> **Evidencia:** La librería expone dos objetos primarios:
>
> - `Series` : Arreglo unidimensional donde cada elemento es trazado por un índice explícito.
> - `DataFrame` : Representación matricial y tabular (2D) con índices en filas y columnas, donde cada columna se comporta como un objeto `Series` .

Conclusión: Comprender estas dos arquitecturas es el prerrequisito vital para manipular bases de datos relacionales, ya que el DataFrame abstrae la lógica de una hoja de cálculo tradicional.

2. Ingesta de Datos (I/O)

PREGUNTA **¿Qué configuración crítica requiere la lectura de archivos delimitados por comas?**

> **Evidencia:** La función `pd.read_csv()` depende de argumentos como `path` (ubicación), `sep` (delimitador), `header` (fila de cabeceras) y `usecols` . Especialmente, `usecols` permite cargar solo las columnas necesarias, y `nrows` limita las filas leídas.

Conclusión: El uso preciso de estos parámetros durante la ingesta no solo evita excepciones de parseo, sino que es una práctica obligatoria para optimizar el consumo de memoria RAM en datasets voluminosos.

PREGUNTA **¿De qué manera facilita Pandas la manipulación de información jerárquica web?**

> **Evidencia:** Con el método `pd.read_json()` , un archivo anidado (común en APIs) se mapea directamente a un `DataFrame` . Posteriormente, puede exportarse a formatos planos usando `df.to_csv("archivo.csv", index=False)` .

Conclusión: Pandas abstrae por completo la iteración manual sobre diccionarios y listas anidadas, transformando la limpieza y conversión de formatos en una operación de una sola línea.

Automatización del Sistema

Módulo Avanzado: Interacción con el OS y Archivos

3. Navegación y Exploración de Entorno

PREGUNTA **¿Cómo inspecciona y localiza un script los recursos dentro de su sistema de archivos?**

> **Evidencia:** Utilizando el módulo `os`. El método `os.getcwd()` rastrea la ruta absoluta donde se ejecuta el script (Current Working Directory). Paralelamente, `os.listdir()` genera una matriz con el nombre de cada archivo y subcarpeta en dicho nivel.

Conclusión: Estas directivas dotan al programa de "consciencia de ubicación", un paso indispensable antes de poder reubicar o alterar cualquier archivo de forma masiva.

4. Lógica de Filtrado y Clasificación

PREGUNTA **¿Cuál es la estrategia óptima para identificar tipos únicos de archivos en un directorio caótico?**

> **Evidencia:** Se combina `os.path.isfile(archivo)` para descartar carpetas, con la extracción del sufijo (`archivo.split(".")[-1]`). Finalmente, al pasar estos strings a un conjunto mediante `set(...)`, se purgan las redundancias.

Conclusión: Intersecar validadores de ruta con estructuras de datos nativas (Set Comprehensions) resulta en un algoritmo elegante que detecta dinámicamente qué carpetas categorizadoras necesitan crearse.

5. Acción: Organizador de Archivos

PREGUNTA **¿De qué forma se garantiza seguridad estructural al mover archivos programáticamente?**

> **Evidencia:** Antes de la traslación, el script evalúa la carencia del directorio destino usando `if not os.path.isdir(tipo): os.mkdir(tipo)`. Para el traslado, ensambla vectores utilizando `os.path.join()` (asegurando barras compatibles) y ejecuta el re-direccionamiento con `os.rename(origen, destino)`.

Conclusión: La delegación del enrutamiento a `os.path.join` hace que el organizador sea 100% multiplataforma. La prevención de fallos mediante evaluaciones lógicas (`isdir`) evita sobreescrituras catastróficas.